

AZURE DATA ENGINEERING JOURNEY

SQL Operators, Aggregates & Clauses

Turning business requirements into SQL solutions — hands on practice with Azure SQL Database and SSMS, solving real world analytical queries using Operators, Aggregate functions and SQL clause.

SWIPE TO LEARN →

· AZURE SQL PRACTICE

From Dataset to Insights

02

Picked up right where we left off last time — same SSMS ⇌ Azure SQL Database connection, this time loaded with a fresh E-Commerce dataset to query in real time.

- ✓ E-Commerce dataset imported into Azure SQL Database
- ✓ 6 logical operators practiced: AND, OR, NOT, IN, BETWEEN, LIKE
- ✓ Aggregate functions covered: SUM, AVG, MAX, MIN, COUNT, DISTINCT
- ✓ Core clauses applied: SELECT, WHERE, GROUP BY, ORDER BY, HAVING, TOP
- ✓ Every single query executed live in SSMS against Azure SQL

```
SELECT * FROM ECOMMERCE
```

	order_id	order_date	customer_id	customer_name	sub-category	category	region	city	quantity	unit_price	discount	payment_method	delivery_days	customer_rating	sales
1	10001	2022-01-01	1102	Aditya Das	Essentials	Beauty	South	Bengaluru	7	373.65	0.28	Wallet	10	4.7	2615.5
2	10002	2022-01-02	1435	Neha Patel	Sarees	Clothing	South	Chennai	7	47.74	0.09	Card	6	3.9	334.18
3	10003	2022-01-03	1860	Rohan Patel	Standard	Beauty	East	Kolkata	3	311.28	0.31	COD	6	2.5	933.84
4	10004	2022-01-04	1270	Priya Gupta	Cameras	Electronics	West	Pune	5	524.47	0.02	Wallet	6	1.6	2622.3
5	10005	2022-01-05	1106	Priya Iyer	T-Shirts	Clothing	West	Pune	5	139.87	0.33	Wallet	4	4.9	699.35
6	10006	2022-01-06	1071	Sneha Singh	Laptops	Electronics	West	Pune	2	264.44	0.2	Card	3	3.3	528.88
7	10007	2022-01-07	1700	Vivaan Gupta	Jeans	Clothing	South	Hyderabad	6	564.6	0.23	Card	1	1.8	3387.6
8	10008	2022-01-08	1020	Aarav Iyer	Standard	Beauty	North	Chandigarh	6	265.71	0.13	Card	2	3.7	1594.2
9	10009	2022-01-09	1614	Ananya Das	Jeans	Clothing	South	Bengaluru	6	224.65	0.27	Card	2	1.4	1347.9
10	10010	2022-01-10	1121	Sneha Sharma	Shoes	Clothing	West	Ahmedabad	4	272.71	0.06	Card	10	4.5	1090.8
11	10011	2022-01-11	1466	Rahul Gupta	Dresses	Clothing	West	Pune	6	318.88	0.02	COD	7	1.4	1913.2
12	10012	2022-01-12	1214	Rohan Singh	Power Banks	Electronics	West	Surat	3	384.71	0.14	COD	11	1.9	1154.1
13	10013	2022-01-13	1330	Ishaan Patel	Premium	Beauty	South	Kochi	7	483.41	0.3	Card	6	1.4	3383.8
14	10014	2022-01-14	1458	Diya Sharma	Jackets	Clothing	West	Mumbai	7	279.48	0.26	COD	8	1.4	1956.3
15	10015	2022-01-15	1087	Aarav Sharma	Shirts	Clothing	East	Ranchi	4	493.23	0.18	Card	9	3.1	1972.9
16	10016	2022-01-16	1372	Kavya Joshi	Jeans	Clothing	West	Ahmedabad	2	327.23	0.2	Wallet	5	3.7	654.46
17	10017	2022-01-17	1099	Aarav Iyer	Sarees	Clothing	North	Jaiour	3	213.01	0.05	Wallet	11	4.2	639.03

Query executed successfully. safadb.database.windows.net... | safa (69) | AZURE_DB | 00:00:04 | 5,000 rows

LOGICAL OPERATORS · AND

03

AND Operator

Combines multiple conditions in a WHERE clause — every condition must be TRUE for a row to qualify. Perfect for narrowing results down to one exact match.

```
-- Customers who bought sarees & paid by card
SELECT CUSTOMER_ID, CUSTOMER_NAME, [SUB-CATEGORY], PAYMENT_METHOD
FROM ECOMMERCE
WHERE [SUB-CATEGORY] = 'SAREES' AND PAYMENT_METHOD = 'CARD'
```

💡 Every extra AND tightens the filter — stack too many and you risk getting zero rows back. Test conditions one at a time when debugging.

```
-- Find customers who purchased sarees and made payment using card
SELECT CUSTOMER_ID, CUSTOMER_NAME, [SUB-CATEGORY], PAYMENT_METHOD FROM ECOMMERCE
WHERE [SUB-CATEGORY]='SAREES' AND PAYMENT_METHOD='CARD'
```

	CUSTOMER_ID	CUSTOMER_NAME	SUB-CATEGORY	PAYMENT_METHOD
1	1435	Neha Patel	Sarees	Card
2	1646	Diya Gupta	Sarees	Card
3	1454	Ishaan Sharma	Sarees	Card
4	1035	Ananya Patel	Sarees	Card
5	1795	Vivaan Das	Sarees	Card
6	1703	Vivaan Iyer	Sarees	Card
7	1446	Vivaan Singh	Sarees	Card
8	1055	Priya Gupta	Sarees	Card
9	1155	Sneha Patel	Sarees	Card
10	1020	Rohan Gupta	Sarees	Card
11	1954	Aditya Gupta	Sarees	Card
12	1231	Neha Reddy	Sarees	Card
13	1328	Ishaan Nair	Sarees	Card
14	1522	Neha Reddy	Sarees	Card
15	1112	Rahul Das	Sarees	Card
16	1282	Neha Kumar	Sarees	Card
17	1629	Ishaan Gupta	Sarees	Card
18	1054	Rohan Patel	Sarees	Card
19	1997	Vivaan Das	Sarees	Card

LOGICAL OPERATORS · OR

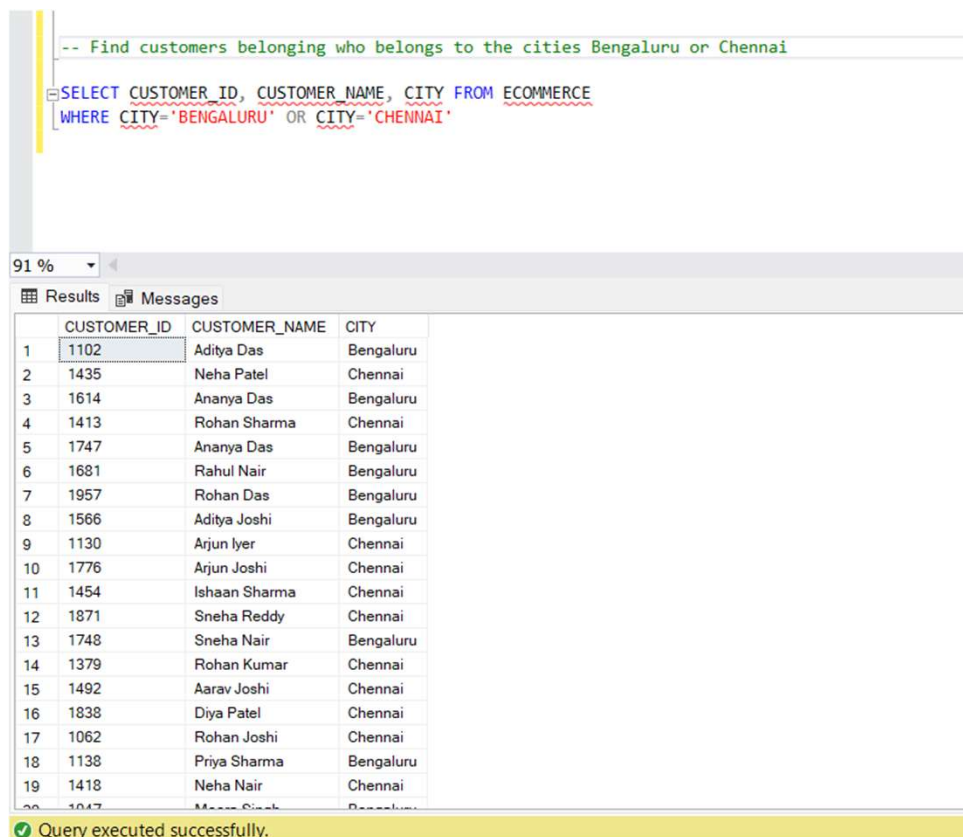
04

OR Operator

Returns a row when at least one condition is TRUE. Where AND narrows results, OR broadens them — useful for pulling multiple cities or segments together.

```
-- Customers who belong to Bengaluru or Chennai
SELECT CUSTOMER_ID, CUSTOMER_NAME, CITY
FROM ECOMMERCE
WHERE CITY = 'BENGALURU' OR CITY = 'CHENNAI'
```

💡 Mixing AND and OR in one query? Wrap conditions in parentheses — SQL evaluates AND before OR, and that order isn't always what you expect.



```
-- Find customers belonging who belongs to the cities Bengaluru or Chennai
SELECT CUSTOMER_ID, CUSTOMER_NAME, CITY FROM ECOMMERCE
WHERE CITY='BENGALURU' OR CITY='CHENNAI'
```

	CUSTOMER_ID	CUSTOMER_NAME	CITY
1	1102	Aditya Das	Bengaluru
2	1435	Neha Patel	Chennai
3	1614	Ananya Das	Bengaluru
4	1413	Rohan Sharma	Chennai
5	1747	Ananya Das	Bengaluru
6	1681	Rahul Nair	Bengaluru
7	1957	Rohan Das	Bengaluru
8	1566	Aditya Joshi	Bengaluru
9	1130	Arjun Iyer	Chennai
10	1776	Arjun Joshi	Chennai
11	1454	Ishaan Sharma	Chennai
12	1871	Sneha Reddy	Chennai
13	1748	Sneha Nair	Bengaluru
14	1379	Rohan Kumar	Chennai
15	1492	Aarav Joshi	Chennai
16	1838	Diya Patel	Chennai
17	1062	Rohan Joshi	Chennai
18	1138	Priya Sharma	Bengaluru
19	1418	Neha Nair	Chennai
20	1047	Mansi Singh	Bengaluru

Query executed successfully.

LOGICAL OPERATORS · NOT

NOT Operator

05

Negates a condition, returning the opposite of what's specified. Paired here with IN, it's a clean way to exclude a whole list of values in a single line.

```
-- Customers outside the South, East & North regions
SELECT ORDER_ID, CUSTOMER_NAME, REGION, CITY
FROM ECOMMERCE
WHERE REGION NOT IN ('SOUTH', 'EAST', 'NORTH')
```

💡 NOT IN silently returns zero rows if the list contains a NULL — worth remembering before you trust an empty result set.

```
--Display customers who are only from the South region.
SELECT ORDER_ID, CUSTOMER_NAME, REGION, CITY FROM ECOMMERCE
WHERE REGION NOT IN ('SOUTH', 'EAST', 'NORTH')
```

91 %

Results Messages

	ORDER_ID	CUSTOMER_NAME	REGION	CITY
1	10004	Priya Gupta	West	Pune
2	10005	Priya Iyer	West	Pune
3	10006	Sneha Singh	West	Pune
4	10010	Sneha Sharma	West	Ahmedabad
5	10011	Rahul Gupta	West	Pune
6	10012	Rohan Singh	West	Surat
7	10014	Diya Sharma	West	Mumbai
8	10016	Kavya Joshi	West	Ahmedabad
9	10019	Ananya Sharma	West	Pune
10	10021	Sneha Gupta	West	Mumbai
11	10023	Arjun Nair	West	Surat
12	10032	Ananya Joshi	West	Pune
13	10035	Ishaan Gupta	West	Ahmedabad
14	10036	Neha Joshi	West	Surat
15	10042	Diya Joshi	West	Pune
16	10048	Diya Gupta	West	Pune
17	10056	Sneha Singh	West	Surat
18	10063	Ishaan Sharma	West	Pune
19	10075	Aarav Joshi	West	Pune
20	10077	Sneha Desai	West	Mumbai

Query executed successfully.

LOGICAL OPERATORS · IN

IN Operator

06

Checks whether a value exists in a list — a shorter, cleaner alternative to stacking multiple OR conditions. Combines easily with AND for richer filters.

```
-- Card/COD payments made from the South region
SELECT CUSTOMER_ID, CUSTOMER_NAME, PAYMENT_METHOD, CITY
FROM ECOMMERCE
WHERE PAYMENT_METHOD IN ('COD', 'Card') AND REGION = 'SOUTH'
```

💡 IN reads like a sentence and scales to long value lists far better than repeating column = value OR column = value. = value.

```
--Find the customers who made the payment using card and cod from south region
SELECT CUSTOMER_ID, CUSTOMER_NAME, PAYMENT_METHOD, CITY
FROM ECOMMERCE
WHERE PAYMENT_METHOD IN ('COD', 'Card') AND REGION='SOUTH'
```

91 %

Results Messages

	ORDER_ID	CUSTOMER_NAME	REGION	CITY
1	10004	Priya Gupta	West	Pune
2	10005	Priya Iyer	West	Pune
3	10006	Sneha Singh	West	Pune
4	10010	Sneha Sharma	West	Ahmedabad
5	10011	Rahul Gupta	West	Pune
6	10012	Rohan Singh	West	Surat
7	10014	Diya Sharma	West	Mumbai
8	10016	Kavya Joshi	West	Ahmedabad
9	10019	Ananya Sharma	West	Pune
10	10021	Sneha Gupta	West	Mumbai
11	10023	Arjun Nair	West	Surat
12	10032	Ananya Joshi	West	Pune
13	10035	Ishaan Gupta	West	Ahmedabad
14	10036	Neha Joshi	West	Surat
15	10042	Diya Joshi	West	Pune
16	10048	Diya Gupta	West	Pune
17	10056	Sneha Singh	West	Surat
18	10063	Ishaan Sharma	West	Pune
19	10075	Aarav Joshi	West	Pune
20	10077	Sneha Singh	West	Mumbai

✓ Query executed successfully.

LOGICAL OPERATORS · BETWEEN

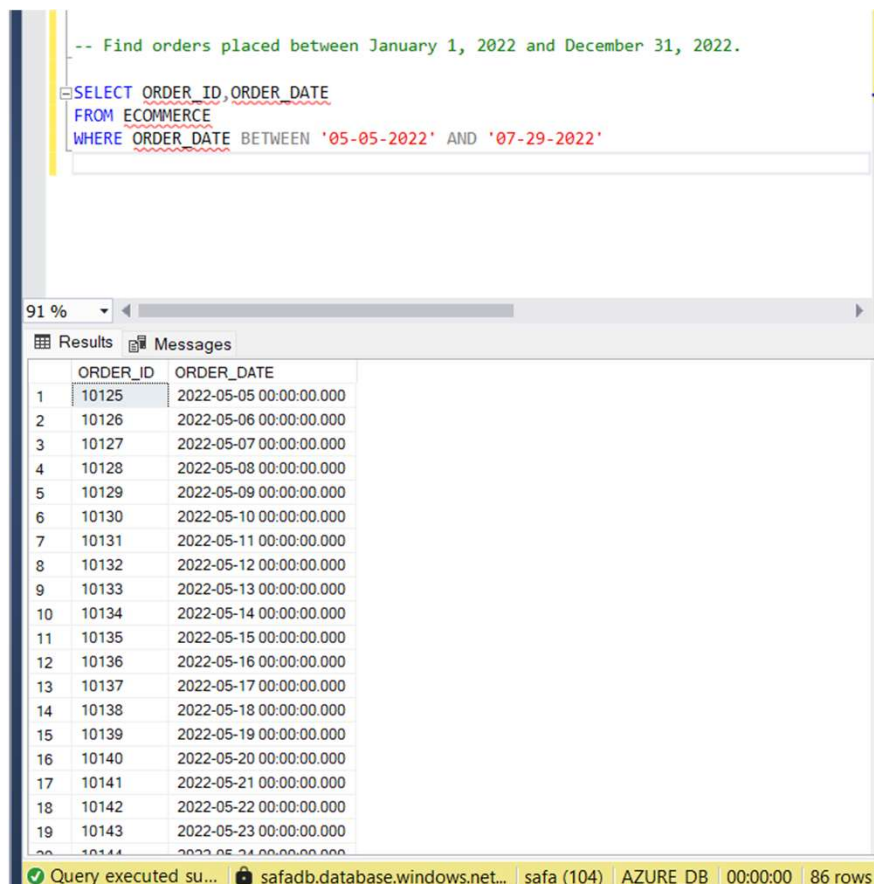
07

BETWEEN Operator

Filters values within a range — inclusive of both endpoints. Works across numbers, dates, and text, and reads more naturally than chaining `>=` and `<=`.

```
-- Orders placed within a fixed date range
SELECT ORDER_ID, ORDER_DATE
FROM ECOMMERCE
WHERE ORDER_DATE BETWEEN '05-05-2022' AND '07-29-2022'
```

💡 BETWEEN is inclusive on both ends — the end date is included in the result, not excluded, so double-check off-by-one check off-by-one assumptions.



```
-- Find orders placed between January 1, 2022 and December 31, 2022.
SELECT ORDER_ID, ORDER_DATE
FROM ECOMMERCE
WHERE ORDER_DATE BETWEEN '05-05-2022' AND '07-29-2022'
```

	ORDER_ID	ORDER_DATE
1	10125	2022-05-05 00:00:00.000
2	10126	2022-05-06 00:00:00.000
3	10127	2022-05-07 00:00:00.000
4	10128	2022-05-08 00:00:00.000
5	10129	2022-05-09 00:00:00.000
6	10130	2022-05-10 00:00:00.000
7	10131	2022-05-11 00:00:00.000
8	10132	2022-05-12 00:00:00.000
9	10133	2022-05-13 00:00:00.000
10	10134	2022-05-14 00:00:00.000
11	10135	2022-05-15 00:00:00.000
12	10136	2022-05-16 00:00:00.000
13	10137	2022-05-17 00:00:00.000
14	10138	2022-05-18 00:00:00.000
15	10139	2022-05-19 00:00:00.000
16	10140	2022-05-20 00:00:00.000
17	10141	2022-05-21 00:00:00.000
18	10142	2022-05-22 00:00:00.000
19	10143	2022-05-23 00:00:00.000
20	10144	2022-05-24 00:00:00.000

Query executed successfully | safadb.database.windows.net... | safa (104) | AZURE_DB | 00:00:00 | 86 rows

LOGICAL OPERATORS · LIKE

LIKE Operator

08

Searches for a text pattern using wildcards. Two placements of the % wildcard below answer two very different real-world questions.

% zero or more characters

_ exactly one character

```
-- Customer names starting with 'NEHA'
SELECT CUSTOMER_NAME
FROM ECOMMERCE
WHERE CUSTOMER_NAME LIKE 'NEHA%'

-- Cities ending with 'BAD'
SELECT CUSTOMER_NAME, CITY
FROM ECOMMERCE
WHERE CITY LIKE '%BAD'
```

💡 LIKE behaves differently across collations — always check a sample of real results before trusting a pattern match at pattern match at scale.

```
--Display all products whose name starts with the letter 'neha'.
SELECT CUSTOMER_NAME FROM ECOMMERCE
WHERE CUSTOMER_NAME LIKE 'NEHA%'
```

CUSTOMER_NAME
1 Neha Patel
2 Neha Singh
3 Neha Joshi
4 Neha Sharma
5 Neha Das
6 Neha Nair
7 Neha Nair
8 Neha Sharma
9 Neha Iyer
10 Neha Nair
11 Neha Iyer
12 Neha Joshi
13 Neha Reddy
14 Neha Das
15 Neha Kumar
16 Neha Reddy
17 Neha Reddy
18 Neha Gupta
19 Neha Patel

Query executed successfully.

```
--Display all customers whose city name end with the letter 'bad'.
SELECT CUSTOMER_NAME, CITY FROM ECOMMERCE
WHERE CITY LIKE '%BAD'
```

CUSTOMER_NAME	CITY
1 Vivaan Gupta	Hyderabad
2 Sneha Sharma	Ahmedabad
3 Kavya Joshi	Ahmedabad
4 Ishaan Gupta	Ahmedabad
5 Rahul Sharma	Hyderabad
6 Sneha Kumar	Hyderabad
7 Meera Joshi	Hyderabad
8 Meera Das	Hyderabad
9 Priya Sharma	Ahmedabad
10 Rohan Patel	Ahmedabad
11 Priya Gupta	Ahmedabad
12 Arjun Das	Ahmedabad
13 Kavya Iyer	Hyderabad
14 Kavya Iyer	Hyderabad
15 Ishaan Gupta	Hyderabad
16 Neha Iyer	Ahmedabad
17 Rohan Singh	Hyderabad
18 Aditya Das	Ahmedabad
19 Arjun Nair	Ahmedabad

Query executed successfully.

AGGREGATE FUNCTIONS · OVERVIEW

09

Six Functions, One Goal

Aggregate functions collapse many rows into a single meaningful number — the backbone of every KPI, dashboard, and business report.

**SUM**

Adds up every value in a column

**AVG**

Calculates the mean value

**MAX**

Returns the highest value

**MIN**

Returns the lowest value

**COUNT**

Counts the number of rows

**DISTINCT**

Lists only unique values

AGGREGATE FUNCTIONS · COUNT

COUNT() Function

10

Counts how many rows match a query — COUNT(*) counts every row including NULLs, making it the fastest way to size up a dataset before deeper analysis.

* counts every row, NULLs included

1 returns a single summary number

```
-- Total number of records in the table  
SELECT COUNT(*) AS TOTAL_COUNT  
FROM ECOMMERCE
```

💡 Use COUNT(column_name) instead of COUNT(*) when you specifically want to ignore NULL values in that column.

```
--Find the total number of rows  
SELECT COUNT(*) AS TOTAL_COUNT FROM ECOMMERCE
```

91 %

Results Messages

	TOTAL_COUNT
1	5000

AGGREGATE FUNCTIONS · SUM

SUM() Function

11

Adds every value in a numeric column into a single total — the first query I run whenever a stakeholder asks, “what's our total revenue?”

⊕ totals every value in the column

∅ NULL values are skipped automatically

```
-- Total sales value across all orders  
SELECT SUM(SALES) AS TOTAL_SALES  
FROM ECOMMERCE
```

💡 SUM() ignores NULL values automatically, so a few missing entries won't silently break your total.

```
-- Find the total sales value  
SELECT SUM(SALES) AS TOTAL_SALES FROM ECOMMERCE
```

91 %

Results Messages

	TOTAL_SALES
1	1248.004

AGGREGATE FUNCTIONS · AVG

AVG() Function

12

Calculates the mean of a numeric column. Wrapping it in ROUND() keeps reports clean — no stakeholder wants a KPI with eight decimal places.



the arithmetic mean of the column



pair with ROUND() for clean output

```
-- Average sales value, rounded to 3 decimals  
SELECT ROUND(AVG(SALES), 3) AS TOTAL_SALES  
FROM ECOMMERCE
```

💡 AVG() also skips NULLs when computing the mean — missing data won't quietly drag your average down.

```
-- Find the average sales value  
SELECT ROUND(AVG(SALES),3) AS TOTAL_SALES FROM ECOMMERCE
```

91 %	
Results Messages	
TOTAL_SALES	
1	1248.004

AGGREGATE FUNCTIONS · DISTINCT

13

DISTINCT Keyword

Strips out duplicate values, leaving only unique entries. It's the fastest way to answer, "how many different categories do we actually sell?"

≠ removes duplicate values

pair with COUNT for a unique tally

```
-- Unique / different categories sold  
SELECT DISTINCT(CITY) AS CITIES  
FROM ECOMMERCE
```

💡 Pair DISTINCT with COUNT — COUNT(DISTINCT CATEGORY) tells you exactly how many unique categories exist in one categories exist in one line.

```
-- Find the unique/different cities  
SELECT DISTINCT(CITY) AS CITIES FROM ECOMMERCE
```

	CITIES
1	Mumbai
2	Chennai
3	Patna
4	Hyderabad
5	Ahmedabad
6	Ranchi
7	Surat
8	Bengaluru
9	Chandigarh
10	Kochi
11	Bhubaneswar
12	Delhi
13	Jaipur
14	Pune
15	Kolkata
16	Lucknow

SQL CLAUSES · WHERE & ORDER BY

14

Filter & Sort Records

WHERE filters rows before they're returned; ORDER BY controls the sequence they arrive in. Together they turn a raw table into a ranked, readable list.

```
-- Display the orders where revenue is greater than 2000
SELECT ORDER_ID, REVENUE
FROM ECOMMERCE
WHERE REVENUE > 2000
ORDER BY REVENUE ASC
```

💡 WHERE runs before ORDER BY — filtering first keeps the sort fast, since SQL never has to rank rows it's about to discard.

```
--Display the orders where revenue is greater than 2000

SELECT ORDER_ID, REVENUE
FROM ECOMMERCE
WHERE REVENUE > 2000
ORDER BY REVENUE ASC
```

	ORDER_ID	REVENUE
1	10122	2000.26
2	14462	2000.76
3	12901	2001.36
4	11514	2001.89
5	14772	2002.15
6	14820	2004.65
7	12114	2006.08
8	11537	2006.24
9	14251	2007.48
10	13936	2008.86
11	12664	2010.08
12	13784	2011.59
13	13869	2011.74
14	13950	2011.83
15	14568	2012.32
16	14782	2014.57
17	11459	2015.48
18	12707	2018.62
19	14076	2021.34
20	14626	2021.66
21	13199	2021.9

Query executed successfully. safadb.database.windows.net... safa (69) AZURE_DB 00:00:00 709 rows

SQL CLAUSES · GROUP BY

GROUP BY Clause

15

Buckets rows that share a value into groups, so an aggregate function can summarize each bucket separately — how a report goes from row-level to category-level.

```
-- Total sales for each sub-category
SELECT [SUB-CATEGORY], SUM(SALES) AS TOTAL_SALES
FROM ORDERS
GROUP BY [SUB-CATEGORY]
```

💡 Every non-aggregated column in SELECT must also appear in GROUP BY — SQL Server throws an error otherwise.

```
--Total sales for each sub-category

SELECT [SUB-CATEGORY], SUM(SALES) AS TOTAL_SALES
FROM ECOMMERCE
GROUP BY [SUB-CATEGORY]
```

110 %

Results Messages

	SUB-CATEGORY	TOTAL_SALES
1	T-Shirts	211668.19
2	Sarees	227675.4
3	Essentials	497231.71
4	Shoes	220855.89
5	Power Banks	260214.47
6	Speakers	318244.69
7	Accessories	499862.33
8	Dresses	239052.79
9	Jackets	253159.81
10	Smart Watches	293288.54
11	Premium	586595.98
12	Jeans	221786.38
13	Standard	545887.87
14	Tablets	268991.97
15	Headphones	288328.95
16	Mobile Phones	286371.92
17	Shirts	233863.95
18	Kurtas	279069.09
19	Cameras	261844.15
20	Laptops	246027.88

✓ Query executed successfully.

SQL CLAUSES · GROUP BY + ORDER BY

16

Rank Grouped Results(ORDER BY)

GROUP BY builds the summary, ORDER BY ranks it — this combination is exactly how you'd answer “which city sells best on average?” in one query.

```
-- Average sales per city, highest first
SELECT CITY, ROUND(AVG(SALES), 2) AS AVG_SALES
FROM ECOMMERCE
GROUP BY CITY
ORDER BY AVG_SALES DESC
```

💡 Sort by the aggregate expression or its alias — ORDER BY works directly on the values GROUP BY just produced.

```
--Find the average sales for each city and display the cities in descending order.
SELECT CITY, ROUND(AVG(SALES), 2) AS AVG_SALES
FROM ECOMMERCE
GROUP BY CITY
ORDER BY AVG_SALES DESC
```

	CITY	AVG_SALES
1	Pune	1316.61
2	Kochi	1304.29
3	Chennai	1294.7
4	Lucknow	1293.9
5	Bhubaneswar	1293.22
6	Mumbai	1282.76
7	Jaipur	1263.36
8	Ranchi	1258.03
9	Bengaluru	1256.91
10	Ahmedabad	1255.6
11	Surat	1235.97
12	Delhi	1225.32
13	Patna	1197.95
14	Kolkata	1190.82
15	Hyderabad	1166.06
16	Chandigarh	1137.49

SQL CLAUSES · TOP

TOP Clause

17

Limits results to a fixed number of rows — paired with ORDER BY, it's the cleanest way to answer “show me my top 5” without scrolling a full table.

```
-- Top 5 cities by total revenue
SELECT TOP 5 CITY, SUM(REVENUE) AS TOTAL_REVENUE
FROM ECOMMERCE
GROUP BY CITY
ORDER BY TOTAL_REVENUE DESC
```

💡 TOP always applies after ORDER BY — SQL Server sorts the full result set first, then slices off the top rows.

```
--Display the top 5 cities with the highest total revenue.

SELECT CITY, SUM(REVENUE) AS TOTAL_REVENUE
FROM ECOMMERCE
GROUP BY CITY
ORDER BY TOTAL_REVENUE DESC

SELECT TOP 5 CITY, SUM(REVENUE) AS TOTAL_REVENUE
FROM ECOMMERCE
GROUP BY CITY
ORDER BY TOTAL_REVENUE DESC
```

	CITY	TOTAL_REVENUE
1	Pune	362660.62
2	Surat	340072.32
3	Mumbai	333515.92
4	Delhi	328495.05
5	Chennai	325098.83
6	Kochi	324455.28
7	Jaipur	322698.37
8	Lucknow	320167.61
9	Ranchi	319320.97
10	Bhubaneswar	317240.12
11	Bangalore	315600.28

	CITY	TOTAL_REVENUE
1	Pune	362660.62
2	Surat	340072.32
3	Mumbai	333515.92
4	Delhi	328495.05
5	Chennai	325098.83

SQL CLAUSES · HAVING

HAVING Clause

18

Filters groups after aggregation. Where WHERE filters individual rows, HAVING filters the summarized results — like “only categories above ₹3,00,000.”

```
-- Categories whose total sales exceed 3,00,000
SELECT CATEGORY, SUM(SALES) AS TOTAL_SALES
FROM ECOMMERCE
GROUP BY CATEGORY
HAVING SUM(SALES) > 500000
ORDER BY SUM(SALES) DESC
```

💡 WHERE can't reference an aggregate like SUM(SALES) — that's exactly the gap HAVING is built to fill.

```
-- Categories whose total sales exceed 5,00,000
SELECT CATEGORY, SUM(SALES) AS TOTAL_SALES
FROM ECOMMERCE
GROUP BY CATEGORY
HAVING SUM(SALES) > 500000
ORDER BY SUM(SALES) DESC
```

110 %

Results Messages

	CATEGORY	TOTAL_SALES
1	Electronics	2223312.57
2	Clothing	1887131.5
3	Home	1196217.97
4	Beauty	933359.92

• SKILL RECAP

19

What I Practiced Today

23 hands-on queries against a live Azure SQL Database — every concept below was written, run, and verified in SSMS on the E-Commerce dataset.



Logical Operators

AND • OR • NOT • IN • BETWEEN • LIKE



Aggregate Functions

SUM • AVG • MAX • MIN • COUNT • DISTINCT



Filtering & Sorting

WHERE • ORDER BY • TOP



Grouping & Summarizing

GROUP BY • HAVING

COMPLETE ✓

Queries That Turned Data into Decisions. 1 Live Database. Real Skills.

Following my Azure Data Engineering journey — one query at a time.
More to come. Stay tuned.

💬 Which SQL clause do you reach for the most in real projects? Let's talk in the comments — and repost if and repost if this helped your revision too!